Numpy

NumPy is a Python library used to perform fast mathematical and numerical operations on large datasets using arrays.

- List → General purpose storage
- NumPy Array → Mathematical and scientific computing
- Stores same data type element

In [3]:
```
# Numpay array
"""creating a numpy array
1)using np.arrray()//1D/2D.
3)using np.Zero/one/empty /randomcs
4)using np.range()
5)using np.linespace()
6)using copy()
7)using idntity()  """
```

Out[3]:
```
'creating a numpy array\n1)using np.arrray()//1D/2D.\n3)using np.Zero/one/emp
ty /randomcs\n4)using np.range()\n5)using np.linespace()\n6)using copy()\n7)u
sing idntity()  '
```

In [4]:
```
import numpy as np
arr1=np.array([1,2,3,4,5])
print("this is my array",arr1)
```

```
this is my array [1 2 3 4 5]
```

In [5]:
```
arr2=np.array([[1,2,3,4],[5,6,7,8]])
print(arr2)
```

```
[[1 2 3 4]
 [5 6 7 8]]
```

In [6]:
```
import numpy as np
arr3=np.zeros((2,3))
print("zero array 2D",arr3)
arr4=np.zeros(3)
print("zero 1D array",arr4)
arr5=np.ones((5,6))
print("onse array",arr5)
arr6=np.identity(5)
print("identity array",arr6)
arr7=np.arange(10)
print("arange array",arr7)
arr8=np.linspace(8,2,8)
arr8
```

```
zero array 2D [[0. 0. 0.]
 [0. 0. 0.]]
zero 1D array [0. 0. 0.]
onse array [[1. 1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1. 1.]
 [1. 1. 1. 1. 1. 1.]]
identity array [[1. 0. 0. 0. 0.]
 [0. 1. 0. 0. 0.]
 [0. 0. 1. 0. 0.]
 [0. 0. 0. 1. 0.]
 [0. 0. 0. 0. 1.]]
arange array [0 1 2 3 4 5 6 7 8 9]
```

Out[6]:
```
array([8.        , 7.14285714, 6.28571429, 5.42857143, 4.57142857,
       3.71428571, 2.85714286, 2.        ])
```

In [7]:
```
import numpy as np
#del print #i use this becouse my kernal tske the print as tuple this is usefu

arr9 = np.linspace(500, 0, 3)
print("THIS is my linspace array it divide array element in proper spacing",ar

arra = np.arange(5, 16, 2)
print("This is my range array, it takes automatically elements:", arra)
arra1=arr3.copy()
print("this is my copy arry it copy arr3",arra1)
```

```
THIS is my linspace array it divide array element in proper spacing [500. 250.
0.]
This is my range array, it takes automatically elements: [ 5  7  9 11 13 15]
this is my copy arry it copy arr3 [[0. 0. 0.]
 [0. 0. 0.]]
```

In [8]:
```
#this use for random element take
import numpy as np

arr = np.random.randint(1, 10, 5)  # from 1 to 9, total 5 numbers
print(arr)
arr5 = np.random.randn(5)
print(arr5)
```

```
[3 2 8 8 6]
[-0.45078929  0.02807017 -1.4014324   0.09192904 -0.06664113]
```

In [9]:
```
#use of reshape wit arange
arr12=np.arange(24).reshape(6,4)
print("thsi is my reshape array",arr12)
```

```
thsi is my reshape array [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
```

In [10]:
```
#attribute & properties of numpy
"""
1)shape
2)ndim
3)size
4)itemsize
5)dtype
7)astype

"""
import numpy as np
arr1=np.array([[1,2,3],[4,5,6],[7,8,9]])
print("check to attribute",arr1)
#use for shape of array
print("this is my shape of array",arr1.shape)
```

```
check to attribute [[1 2 3]
 [4 5 6]
 [7 8 9]]
this is my shape of array (3, 3)
```

In [11]:
```
print("use for find dimension of array ",arr1.ndim)
print("use for find element number present of array ",arr1.size)
print("use for find list size of array",arr1.itemsize)
print("use for find data type and byte size",arr1.dtype)
print("int to float:", arr1.astype(float))
print("Foat to int:", arr1.astype(int))
```

```
use for find dimension of array  2
use for find element number present of array  9
use for find list size of array 8
use for find data type and byte size int64
int to float: [[1. 2. 3.]
 [4. 5. 6.]
 [7. 8. 9.]]
Foat to int: [[1 2 3]
 [4 5 6]
 [7 8 9]]
```

In [12]:
```
# our new topic is why we prafer list over array
"""
1)less memory
2)fast
3)convenient
"""
#1)less memory
import sys
import numpy as np

my_list = list(range(100))   # normal Python list
arr10 = np.arange(100)       # NumPy array

print("my list size..!", sys.getsizeof(my_list))
print("my array size..!", arr10.itemsize * arr10.size)
```

```python
print("for fast and convenient")
import time

x = range(100000)
y = range(100000, 200000)

start_time = time.time()

# Your code to execute
print("This is example program")

end_time = time.time()

print("list Execution time:", end_time - start_time, "seconds")


import numpy as np
import time

print("For execution of array time")

a = np.arange(100000)  # values 0 to 99999
b = np.arange(100000, 200000)  # values 100000 to 199999

start_time = time.time()

c = a + b

end_time = time.time()

print("Array Execution time:", end_time - start_time, "seconds")

"""
in our observetion array take lees memory
array have a fast execution speed
and convenient is easey in arry like 'c=a+b'
"""
```

```
my list size..! 856
my array size..! 800
for fast and convenient
This is example program
list Execution time: 7.152557373046875e-05 seconds
For execution of array time
Array Execution time: 0.0007295608520507812 seconds
```

Out[12]: "\nin our observetion array take lees memory\narray have a fast execution speed\nand convenient is easey in arry like 'c=a+b'\n"

In [13]:
```python
#next topic is index slicing and iteration
import numpy as np

arry = np.array([1, 3, 5, 7, 9])
print("My array is ..!", arry)
```

```
print(arry[2])
print(arry[1:4])
arry1=np.array([[1,2,4],[7,5,1],[5,4,6]])
print(arry1)
```

```
My array is ..! [1 3 5 7 9]
5
[3 5 7]
[[1 2 4]
 [7 5 1]
 [5 4 6]]
```

In [14]:
```
arry1[:2]
```

Out[14]:
```
array([[1, 2, 4],
       [7, 5, 1]])
```

In [15]:
```
arry1[1]
```

Out[15]:
```
array([7, 5, 1])
```

In [16]:
```
arry1[ :2:4]
```

Out[16]:
```
array([[1, 2, 4]])
```

In [17]:
```
#iteration
for i in arry:
   print(i)
```

```
1
3
5
7
9
```

In [18]:
```
for i in np.nditer(arry1):
    print(i)
```

```
1
2
4
7
5
1
5
4
6
```

In [19]:
```
#reshaping numpy arry
"""
1)ravel (use to convert 2D,3D,4D...nD to 1D)
2)transpose () (When you have a 2-D array (matrix), rows become columns and co
                 This operation is called Transpose
3)stacking
```

```
4)splitting  Splitting (Dividing arrays)We can split a big array into small or
5)reshape  (Changing array shape)But total number of elements must be same!
"""
#ravel 2D to 1D convert
arry1=np.array([[1,2,4],[7,5,1],[5,4,6]])
print(arry1)
print("my ravel array \n",arry1.ravel())
#transpose use row to column and column to row
print("my transpose array \n",arry1.transpose())

#stacking thare are two type of stackin horizantal and vartical & use to combi
```

```
[[1 2 4]
 [7 5 1]
 [5 4 6]]
my ravel array
 [1 2 4 7 5 1 5 4 6]
my transpose array
 [[1 7 5]
 [2 5 4]
 [4 1 6]]
```

In [20]:

```
#stacking thare are two type of stackin horizantal and vartical & use to combi
a=np.array([[1,2,4],[6,7,8]])
b=np.array([[7,8,1],[3,1,7]])
print("this is a array\n",a)
print("horizental \n",np.hstack((a,b)))
print("vartical \n",np.vstack((a,b)))
np.hstack(a)

#splite in array
```

```
this is a array
 [[1 2 4]
 [6 7 8]]
horizental
 [[1 2 4 7 8 1]
 [6 7 8 3 1 7]]
vartical
 [[1 2 4]
 [6 7 8]
 [7 8 1]
 [3 1 7]]
```

Out[20]:

```
array([1, 2, 4, 6, 7, 8])
```

In [21]:

```
#splite in array
a = np.array([[1, 2, 4],
              [6, 7, 8]])

b = np.array([[7, 8, 1],
              [3, 1, 7]])

print(np.hsplit(a, [2]))
print(np.vsplit(a, [2]))
```

```
[array([[1, 2],
       [6, 7]]), array([[4],
       [8]])]
[array([[1, 2, 4],
       [6, 7, 8]]), array([], shape=(0, 3), dtype=int64)]
```

In [22]:
```
#indexing wint boolean array
arr=np.random.randint(low=1,high=100,size=20).reshape(4,5)
arr
```

Out[22]:
```
array([[14, 16, 59, 10, 48],
       [60, 61, 73, 81, 84],
       [94, 47, 54, 15, 81],
       [43, 19, 48, 54, 48]], dtype=int32)
```

In [23]:
```
print("he print all row 0 index\n",arr[0])
```

```
he print all row 0 index
 [14 16 59 10 48]
```

In [24]:
```
print("it show 0row 2 position element\n",arr[0][2])
```

```
it show 0row 2 position element
 59
```

In [25]:
```
print(arr>50)
print("in integer element\n it only pass true lelment of array\n",arr[arr>50])
#suppose i have to put condition on array we can so as follow
print("it search less than 50 and not divided by 2\n",arr[(arr>50)&(arr/2 !=0)
```

```
[[False False  True False False]
 [ True  True  True  True  True]
 [ True False  True False  True]
 [False False False  True False]]
in integer element
 it only pass true lelment of array
 [59 60 61 73 81 84 94 54 81 54]
it search less than 50 and not divided by 2
 [59 60 61 73 81 84 94 54 81 54]
```

In [ ]:

In [22]:
```
import numpy as np
a=np.random.randint(10,100,14)
print(a)
np.random.seed(1)
```

```
[98 22 32 81 38 20 90 71 36 35 97 42 14 87]
```

In [37]:
```
np.random.seed(22)#with the help of seed we fix one output for multipal time r
b=np.random.rand(10)
print(a)
```

```
[98 22 32 81 38 20 90 71 36 35 97 42 14 87]
```

In [39]:
```
c=np.random.rand(10)#in this line when we run code value change every time
c
```

Out[39]:
```
array([0.76799274, 0.68821832, 0.38718348, 0.61520583, 0.42755524,
       0.58428964, 0.70263552, 0.11189517, 0.92326993, 0.98888627])
```

In [6]:
```
import numpy as np
np.random.seed()#in this function we fix output valu.
d=np.random.rand(10)
print(d)
```

```
[0.49949634 0.04929597 0.67813691 0.32204578 0.90928033 0.10284345
 0.04136832 0.70608385 0.76678229 0.33490802]
```

In [3]:
```
import numpy as np
"""e = np.random.randint(10).reshape(2, 5)
print(e)"""
e = np.random.randint(10, size=10).reshape(2, 5)
print(e)
```

```
[[3 9 9 0 7]
 [7 2 8 3 9]]
```

In [22]:
```
import numpy as np
np.random.seed(20)
print(np.random.randint(1, 50, 5))
```

```
[45 39 22 18  6]
```

In [20]:
```
import numpy as np
a1=np.arange(10)
a1
a2=np.arange(12).reshape(3,4)
a2
a3=np.arange(8).reshape(2,2,2)
a3
```

Out[20]:
```
array([[[0, 1],
        [2, 3]],

       [[4, 5],
        [6, 7]]])
```

In [31]:
```
print(a2)
a2[2,3]
print("hear i access the second row third colum element..=",a2[1,2])
#the first number for row and second number for column
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
hear i access the second row third colum element..= 6
```

In [24]:
```
a3
```

Out[24]:
```
array([[[0, 1],
        [2, 3]],

       [[4, 5],
        [6, 7]]])
```

In [35]:
```
print("this is my array element=",a3[0,1,0],"this is my second element=",a3[1,
# whe we retrive one element retrive from array we use comma ,
"""this is 3D array in this array two 2D array is present first number for whi
second element for row and third element for column."""
```

```
this is my array element= 2 this is my second element 5
```

Out[35]:
```
'this is 3D array in this array two 2D array is present first number for whic
h 2D array element is present.\nsecond element for row and third element for
column.'
```

In [41]:
```
print(a1)
a1[1:6]
```

```
[0 1 2 3 4 5 6 7 8 9]
```

Out[41]:
```
array([1, 2, 3, 4, 5])
```

In [73]:
```
print(a2)
print(a2[0])# this is use for first full row retrive
print("\n",a2[:,2])
print("\n",a2[1:,1:3])
print("\nfor 0,8,3,11 element=",a2[::2,::3])
print("\nfor 1,3,9,11 element=",a2[::2,1::2])
print("\nfor 4,7 element=",a2[1,::3])
print("\nfor 1,2,3,5,6,7 element=",a2[:2,1:])
print("\nfor 1,3,5,7 element=",a2[:2,1::2])
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
[0 1 2 3]

 [ 2  6 10]

 [[ 5  6]
 [ 9 10]]

for 0,8,3,11 element= [[ 0  3]
 [ 8 11]]

for 1,3,9,11 element= [[ 1  3]
 [ 9 11]]

for 4,7 element= [4 7]

for 1,2,3,5,6,7 element= [[1 2 3]
 [5 6 7]]

for 1,3,5,7 element= [[1 3]
 [5 7]]
```

In [98]:

```python
# lets pratics in a3 which is 3D array
a3=np.arange(27).reshape(3,3,3)
print(a3)
print("i have to print secode array\n",a3[::2])
print("\n i have to first array second row",a3[0,1,:])
print("\n i have to second array second column",a3[1,:,1])
print("\n i have to third array 22,23,25,26",a3[2,1:,1:])
print("\n i have to first array 0,2,and third array 18,20",a3[::2,0,::2])
```

```
[[[ 0  1  2]
  [ 3  4  5]
  [ 6  7  8]]

 [[ 9 10 11]
  [12 13 14]
  [15 16 17]]

 [[18 19 20]
  [21 22 23]
  [24 25 26]]]
i have to print secode array
 [[[ 0  1  2]
  [ 3  4  5]
  [ 6  7  8]]

 [[18 19 20]
  [21 22 23]
  [24 25 26]]]

 i have to first array second row [3 4 5]

 i have to second array second column [10 13 16]

 i have to third array 22,23,25,26 [[22 23]
 [25 26]]

 i have to first array 0,2,and third array 18,20 [[ 0  2]
 [18 20]]
```

In [112…

```python
#topic is Iterating in Python
 #hear i print 1D array with iteration
import numpy as np
print("my 1D array\n", a1)
for i in a1:

    print(i)
    #hear i print 3D array with iteration

print("my 3D array\n",a3)
```

```
my 1D array
 [0 1 2 3 4 5 6 7 8 9]
hear i print me my 2D array
0
hear i print me my 2D array
1
hear i print me my 2D array
2
hear i print me my 2D array
3
hear i print me my 2D array
4
hear i print me my 2D array
5
hear i print me my 2D array
6
hear i print me my 2D array
7
hear i print me my 2D array
8
hear i print me my 2D array
9
my 2D array
 [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]

[0 1 2 3]
[4 5 6 7]
[ 8  9 10 11]
my 3D array
 [[[ 0  1  2]
  [ 3  4  5]
  [ 6  7  8]]

 [[ 9 10 11]
  [12 13 14]
  [15 16 17]]

 [[18 19 20]
  [21 22 23]
  [24 25 26]]]
```

In [116…

```python
 #hear i print 2D array with iteration
print(a2)
print("hear i print me my 2D array")
for i in a2:
    print(i)
```

```
[[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
hear i print me my 2D array
[0 1 2 3]
[4 5 6 7]
[ 8  9 10 11]
```

In [117…

```python
 #hear i print 3D array with iteration
print(a3)
print("hear i print me my 3D array")

for i in a3:
    print(i)
    #suppose i have to print this 3D array in  1D
```

```
[[[ 0  1  2]
  [ 3  4  5]
  [ 6  7  8]]

 [[ 9 10 11]
  [12 13 14]
  [15 16 17]]

 [[18 19 20]
  [21 22 23]
  [24 25 26]]]
hear i print me my 3D array
[[0 1 2]
 [3 4 5]
 [6 7 8]]
[[ 9 10 11]
 [12 13 14]
 [15 16 17]]
[[18 19 20]
 [21 22 23]
 [24 25 26]]
```

In [119…

```python
#hear i printing array in one line
print("converting 3D array in to 1D array and print ")
for  i in np.nditer(a3):
    print(i)
```

```
converting 3D array in to 1D array and print
0
1
2
3
4
5
6
7
8
9
10
11
12
13
14
15
16
17
18
19
20
21
22
23
24
25
26
```

In [137…

```python
# topic is reshap,transpose and ravel.
#reshape
a5=np.arange(12).reshape(6,2)# it reshaping the 1D array by colum and row wise
print(a5)

#transpose
np.transpose(a5)# it use to convert row into colum and column into row
# other syntax of transpose is
print("thia is other systax of transpose",a3.T)
# nex topic is ravel
print("\nthis ravel is use for convert 2D...nD array into 1D")
np.ravel(a3)# hare i convert 3D array into 1D array
```

```
[[ 0  1]
 [ 2  3]
 [ 4  5]
 [ 6  7]
 [ 8  9]
 [10 11]]
thia is other systax of transpose [[[ 0  9 18]
  [ 3 12 21]
  [ 6 15 24]]

 [[ 1 10 19]
  [ 4 13 22]
  [ 7 16 25]]

 [[ 2 11 20]
  [ 5 14 23]
  [ 8 17 26]]]

this ravel is use for convert 2D...nD array into 1D
```

Out[137…

```
array([ 0,  1,  2,  3,  4,  5,  6,  7,  8,  9, 10, 11, 12, 13, 14, 15, 16,
       17, 18, 19, 20, 21, 22, 23, 24, 25, 26])
```

In [146…

```
#Next topic is Stacking and splitting
"""1) my first topic is stacking is use for concatinating of two array.
   2) but it has condition array shoud in 2*2,3*3...n*n.
   3)two type of stacking first is horizental and vartical.
   4) if i do horizenat of 2*2 it output in 2*4
   5)if i do vartical stack of 2*2 output in 4*2 """
a4=np.arange(12).reshape(3,4)
print("this is a4=\n",a4)
a5=np.arange(12,24).reshape(3,4)
print("this is a5=\n",a5)
print("printing the operation of vstack and hstack")
print("\n",np.hstack((a4,a5))) #hstack use for horizental print
print("\n",np.vstack((a4,a5))) #vstack use for vartical print
```

```
this is a4=
 [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
this is a5=
 [[12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
printing the operation of vstack and hstack

 [[ 0  1  2  3 12 13 14 15]
 [ 4  5  6  7 16 17 18 19]
 [ 8  9 10 11 20 21 22 23]]

 [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]
 [12 13 14 15]
 [16 17 18 19]
 [20 21 22 23]]
```

In [158…

```
#topic is Splitting
""" this spliting is perform on one array it brake array in horizental and var
"""
print("my operation array\n",a4)
print("this is use for partication of array\n",np.hsplit(a4,2))
print("this is use for partication of array\n",np.vsplit(a5,3))#vartical type
```

```
my operation array
 [[ 0  1  2  3]
 [ 4  5  6  7]
 [ 8  9 10 11]]
this is use for partication of array
 [array([[0, 1],
       [4, 5],
       [8, 9]]), array([[ 2,  3],
       [ 6,  7],
       [10, 11]])]
this is use for partication of array
 [array([[12, 13, 14, 15]]), array([[16, 17, 18, 19]]), array([[20, 21, 22, 2
3]])]
```

In [10]:

```
#fancy indexing and boolean indexing
import numpy as np
a = np.random.randint(1,100,24).reshape(6,4)
print(a)
print("we learn about fancy indexing")
""" if i want first row ,3,6 row how i feach then we use fancy indexing where
    we pass list and put index in list of row which i want print"""
print("this is for row printing",a[[0,2,4,5]])
print("\nthis is for column print\n",a[:,[0,2,3]])  #to print column
```

```
[[60  5 93 30]
 [62 16 36 76]
 [13 85 91 65]
 [ 7  9 74 78]
 [48 67 91  6]
 [94 62 75 85]]
we learn about fancy indexing
this is for row printing [[60  5 93 30]
 [13 85 91 65]
 [48 67 91  6]
 [94 62 75 85]]

this is for column print
 [[60 93 30]
 [62 36 76]
 [13 91 65]
 [ 7 74 78]
 [48 91  6]
 [94 75 85]]
```

In [31]:

```
#boolean indexing
a = np.random.randint(1,100,24).reshape(6,4)
print(a)
#find all numbers greater than 50
print("this is boolean array\n",a>50) #in combination of true and false this i
print("this is mask array\n",a[a>50])#this is mask array it filter the valu ar
print("this is even number of array\n",a[a%2==0])#find even number
print("find all number greater than 50 and are even",a[(a>50) & (a%2==0)])
print("\nfind all number not divisible by 7\n",a [a%7 !=0])
print("\nfind all number not divisible by 7\n",a [~(a%7 ==0)])
```

```
[[94 93 18 25]
 [81 34 60 13]
 [ 3 68 44 37]
 [32 62 46 81]
 [52 56 41 70]
 [ 2 17 96  6]]
this is boolean array
 [[ True  True False False]
 [ True False  True False]
 [False  True False False]
 [False  True False  True]
 [ True  True False  True]
 [False False  True False]]
this is mask array
 [94 93 81 60 68 62 81 52 56 70 96]
this is even number of array
 [94 18 34 60 68 44 32 62 46 52 56 70  2 96  6]
find all number greater than 50 and are even [94 60 68 62 52 56 70 96]

find all number not divisible by 7
 [94 93 18 25 81 34 60 13  3 68 44 37 32 62 46 81 52 41  2 17 96  6]

find all number not divisible by 7
 [94 93 18 25 81 34 60 13  3 68 44 37 32 62 46 81 52 41  2 17 96  6]
```

In [35]:

```
#Broadcasting
"""the term brodcasting describes how numpy treats arrys with different shapes
arithmetic operations.
    the smaller array is "broadcast" across the larger array so that they have
    make the two array have the same number of dimensions."""
#same shape for broadcasting like 3*6,6*3 or 2*3,3*2 like that
a=np.arange(6).reshape(2,3)
b= np. arange(6,12).reshape(2,3)
print(a)
print(b)
print("this is my array addition\n",a+b)
```

```
[[0 1 2]
 [3 4 5]]
[[ 6  7  8]
 [ 9 10 11]]
this is my array addition
 [[ 6  8 10]
 [12 14 16]]
```

In [74]:

```
# diff shape  this is callad broadcasting
a=np.arange(6).reshape(2,3)
b= np. arange(3).reshape(1,3)
print(a)
print(b)
print("this is my array addition\n",a+b)
#three basic ruls of broadcasting 3+1=9 element array
arra = np.arange(8)
arra2 = np.array([8])
```

```
print("my addition is\n",arra+arra2)# when i do addition i get arra

#second 9+3=9
i=np.arange(9).reshape(3,3)
j=np.array([5,3,4])
print("this is second rull\n",i+j)
#third rull is column 3 element and row0 index 3 element row
y=np.arange(3).reshape((3,1)) #3row 1column
z=np.arange(3) #3 column
print("this is third y array\n",y)
print("this is third z array\n",z)
print("addition of y+z is\n=",y+b)
```

```
[[0 1 2]
 [3 4 5]]
[[0 1 2]]
this is my array addition
 [[0 2 4]
 [3 5 7]]
my addition is
 [ 8  9 10 11 12 13 14 15]
this is second rull
 [[ 5  4  6]
 [ 8  7  9]
 [11 10 12]]
this is third y array
 [[0]
 [1]
 [2]]
this is third z array
 [0 1 2]
addition of y+z is
= [[0 1 2]
 [1 2 3]
 [2 3 4]]
```

In [99]:

```
"""working with mathmatical formulas"""
# sigmoid
def sigmoid(array):
    return 1/(1+np.exp(-(array)))
a=np.arange(10)
sigmoid(a)
#mean squared error
actual=np.random.randint(1,50,25)

predicted=np.random.randint(1,50,25)
def mse(actual,predicted):
    return np.mean((actual - predicted)**2)
    print(mse)

mse(actual,predicted)


#categorocal cross emtropy
```

Out[99]: np.float64(422.96)

In [109…

```
#Working with missing valuse

import numpy as np

a = np.array([1, 2, 3, 4, np.nan, 6])#nan is a null valu
print(a)
print("this is without null function"a[~np.isnan(a)]
```

```
[ 1.  2.  3.  4. nan  6.]
```

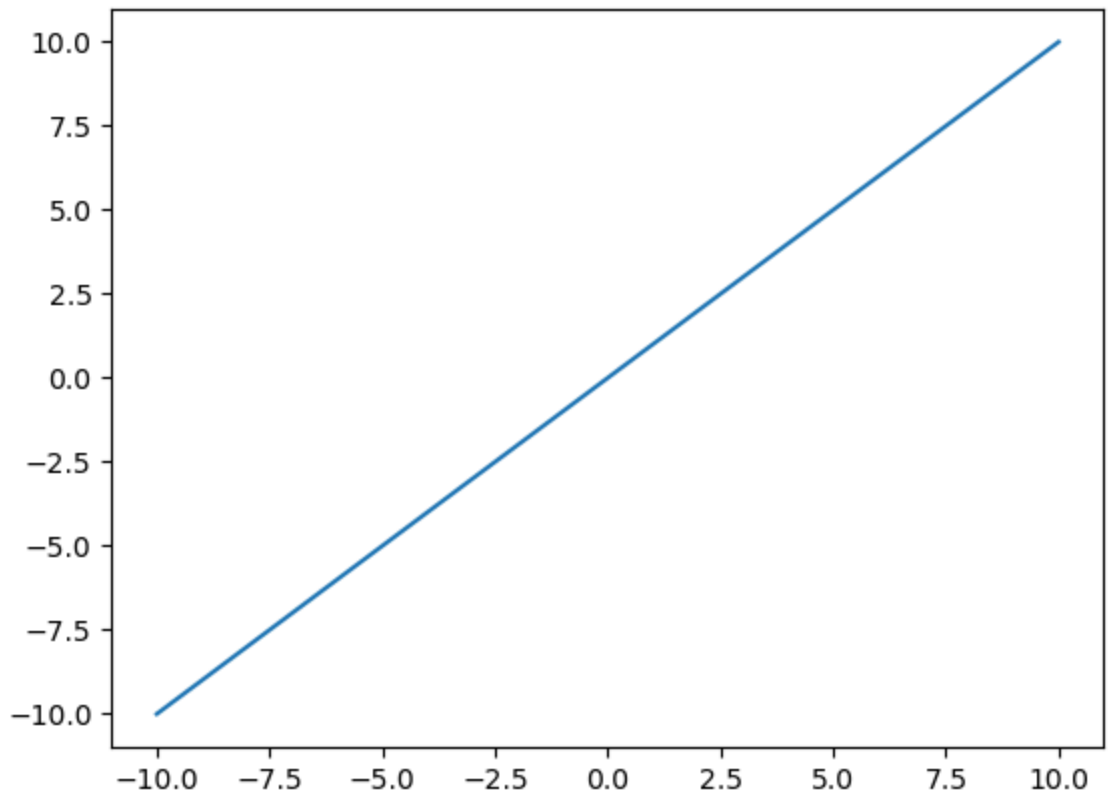
Out[109… array([1., 2., 3., 4., 6.])

In [114…

```
#Plotting graphs
#x=y
import matplotlib.pyplot as plt
x=np.linspace(-10,10,100)
print(x)
y=x
plt.plot(x,y)
```

```
[-10.          -9.7979798   -9.5959596   -9.39393939  -9.19191919
  -8.98989899  -8.78787879  -8.58585859  -8.38383838  -8.18181818
  -7.97979798  -7.77777778  -7.57575758  -7.37373737  -7.17171717
  -6.96969697  -6.76767677  -6.56565657  -6.36363636  -6.16161616
  -5.95959596  -5.75757576  -5.55555556  -5.35353535  -5.15151515
  -4.94949495  -4.74747475  -4.54545455  -4.34343434  -4.14141414
  -3.93939394  -3.73737374  -3.53535354  -3.33333333  -3.13131313
  -2.92929293  -2.72727273  -2.52525253  -2.32323232  -2.12121212
  -1.91919192  -1.71717172  -1.51515152  -1.31313131  -1.11111111
  -0.90909091  -0.70707071  -0.50505051  -0.3030303   -0.1010101
   0.1010101    0.3030303    0.50505051   0.70707071   0.90909091
   1.11111111   1.31313131   1.51515152   1.71717172   1.91919192
   2.12121212   2.32323232   2.52525253   2.72727273   2.92929293
   3.13131313   3.33333333   3.53535354   3.73737374   3.93939394
   4.14141414   4.34343434   4.54545455   4.74747475   4.94949495
   5.15151515   5.35353535   5.55555556   5.75757576   5.95959596
   6.16161616   6.36363636   6.56565657   6.76767677   6.96969697
   7.17171717   7.37373737   7.57575758   7.77777778   7.97979798
   8.18181818   8.38383838   8.58585859   8.78787879   8.98989899
   9.19191919   9.39393939   9.5959596    9.7979798   10.        ]
```

Out[114… [<matplotlib.lines.Line2D at 0x26f54edd310>]

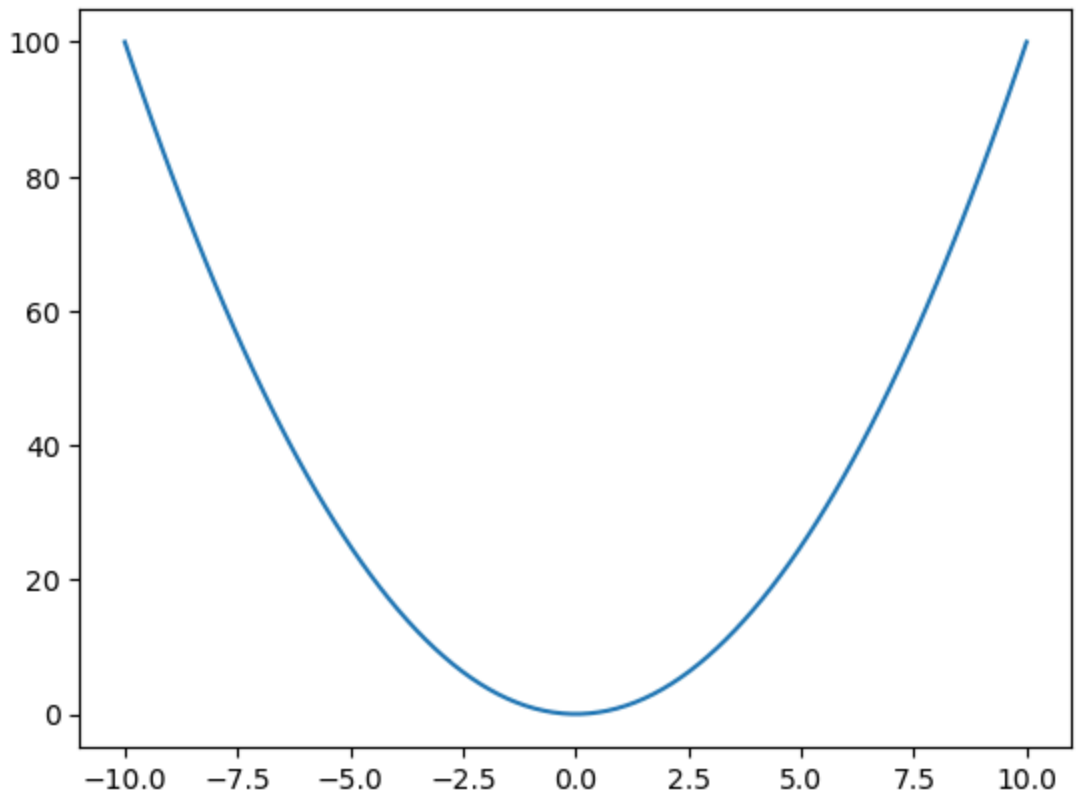


In [116…

```
#parabola
#y=y = x^2
x=np.linspace(-10,10,100)
y=x**2
plt.plot(x,y)
```

Out[116…

```
[<matplotlib.lines.Line2D at 0x26f54f62850>]
```

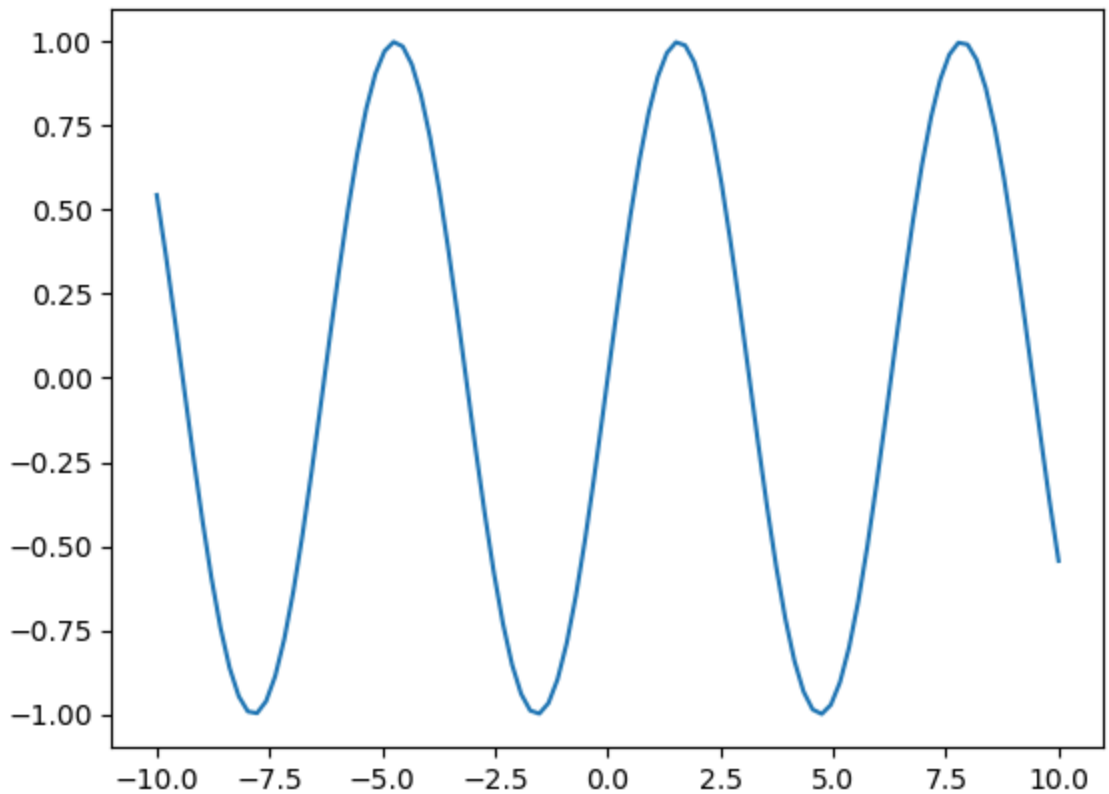


In [119…

```
#y=sin(x)       sinx graph
a=np.linspace(-10,10,100)
y=np.sin(x)
plt.plot(x,y)
```

Out[119…

```
[<matplotlib.lines.Line2D at 0x26f55185d10>]
```



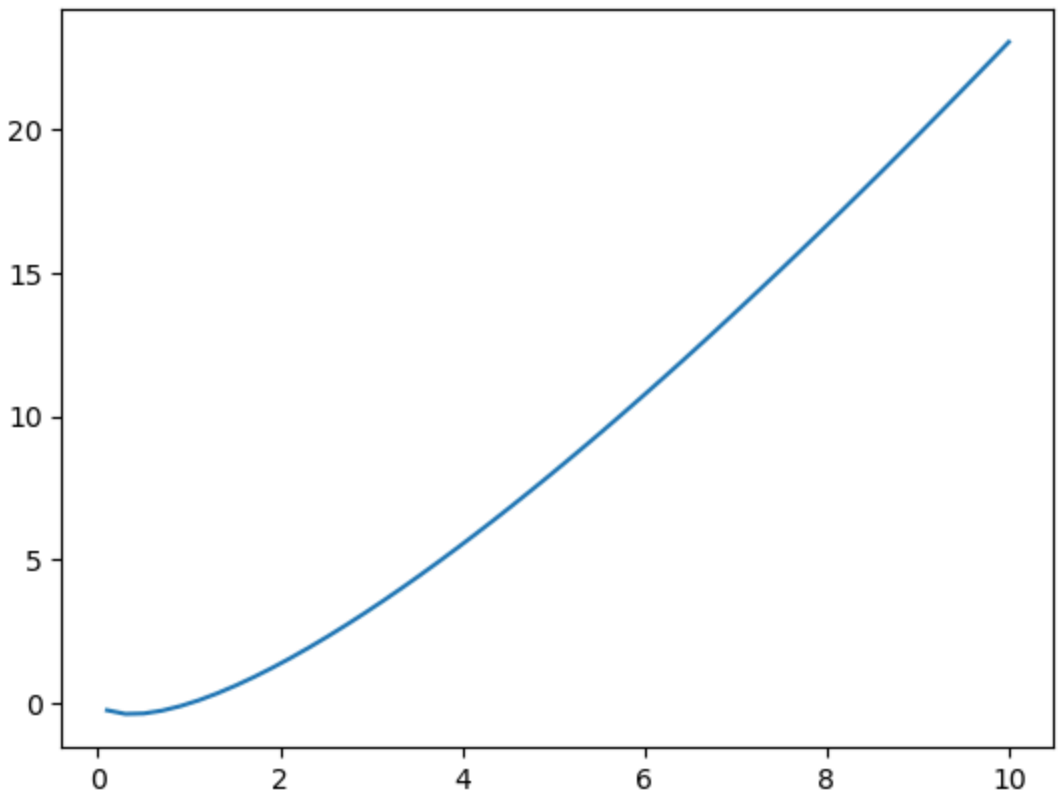
In [125…

```
# y= xlog(x)
x=np.linspace(-10,10,100)
y=x* np.log(x)
plt.plot(x,y)
```

```
C:\Users\jivankakde\AppData\Local\Temp\ipykernel_8348\2859134236.py:3: RuntimeW
arning: invalid value encountered in log
  y=x* np.log(x)
```

Out[125…

```
[<matplotlib.lines.Line2D at 0x26f554d3610>]
```

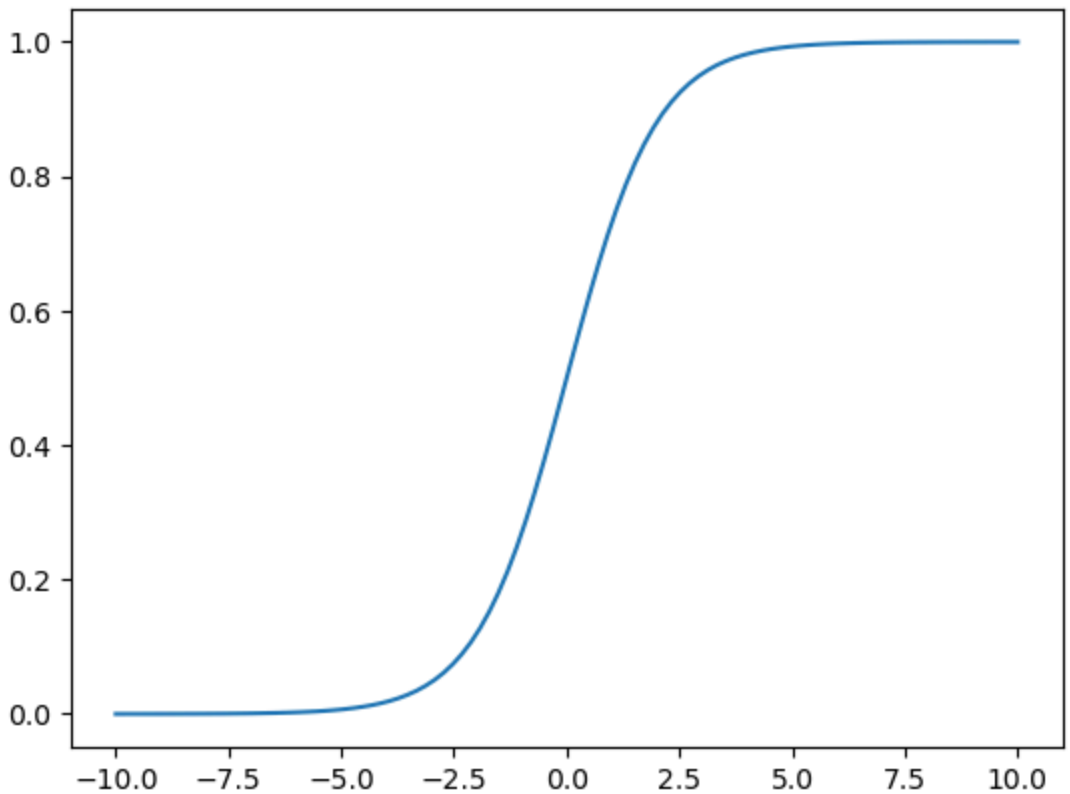


In [126…

```
# sigmoid
x=np.linspace(-10,10,100)
y=1/(1+np.exp(-x))
plt.plot(x,y)
```

Out[126…

```
[<matplotlib.lines.Line2D at 0x26f55548cd0>]
```



In [7]:
```
#sorting in array
import numpy as np
a1=np.random.randint(1,100,15)
a1
```

Out[7]:
```
array([45,  3,  6, 87, 96, 10, 20, 56, 67, 26, 23, 67, 69, 69, 21],
      dtype=int32)
```

In [8]:
```
np.sort(a1) # for 1D array
```

Out[8]:
```
array([ 3,  6, 10, 20, 21, 23, 26, 45, 56, 67, 67, 69, 69, 87, 96],
      dtype=int32)
```

In [16]:
```
b=np.random.randint(1,100,24).reshape(6,4)
print(b)
print("\n",np.sort(b),"\n")# for 2D array and this array sort by row wise
#if i want sort by row wise it use axis=0
# if  i want sort by  column wise we use axis==1
print("array sort by column wise\n",np.sort(b,axis=0))
```

```
[[78 57 28 35]
 [61 10 66 72]
 [64 66 47 80]
 [59  9 62 95]
 [13 15 70 85]
 [82 86 88 29]]

 [[28 35 57 78]
 [10 61 66 72]
 [47 64 66 80]
 [ 9 59 62 95]
 [13 15 70 85]
 [29 82 86 88]]

array sort by column wise
 [[13  9 28 29]
 [59 10 47 35]
 [61 15 62 72]
 [64 57 66 80]
 [78 66 70 85]
 [82 86 88 95]]
```

In [19]:

```
# np.append
#numpy.append() appends values along the mentioned axis at the end of the arra
print(a1)
#if i want add element in this array i use append
print("for 1D array\n",np.append(a1,200),"\n")
""" if i want add 2D array column contain 1. """
np.append(b,np.ones((b.shape[0],1)),axis=1)
```

```
[45  3  6 87 96 10 20 56 67 26 23 67 69 69 21]
for 1D array
 [ 45   3   6  87  96  10  20  56  67  26  23  67  69  69  21 200]
```

Out[19]:

```
array([[78., 57., 28., 35.,  1.],
       [61., 10., 66., 72.,  1.],
       [64., 66., 47., 80.,  1.],
       [59.,  9., 62., 95.,  1.],
       [13., 15., 70., 85.,  1.],
       [82., 86., 88., 29.,  1.]])
```

In [24]:

```
# np.concatenate
#numpy.concatenate() function comcatenate a squencee of arrays along an existi
c= np.arange(6).reshape(2,3)
d= np.arange(6,12).reshape(2,3)
np.concatenate((c,d),axis=0) #if i want concatenate with colunm wise i use axi
#this function is replacement of hstack,vstacking
```

Out[24]:

```
array([[ 0,  1,  2],
       [ 3,  4,  5],
       [ 6,  7,  8],
       [ 9, 10, 11]])
```

In [26]:

```
#np.unique
```

```
# with the help of np.unique() method we get the unique values from an array g
e=np.array([1,1,2,2,3,3,4,4,5,5,6,6])
np.unique(e)
```

Out[26]:
```
array([1, 2, 3, 4, 5, 6])
```

In [40]:
```
# np.expand_dim
#with te help of this function we can get the expanded dimensions of an array
print("this is my 1D array",a1,"\nshape of array is",a1.shape)

print(np.expand_dims(a1,axis=0),"\nthis is my a1 array shape=",np.expand_dims(
```

```
this is my 1D array [45  3  6 87 96 10 20 56 67 26 23 67 69 69 21]
shape of array is (15,)
[[45  3  6 87 96 10 20 56 67 26 23 67 69 69 21]]
this is my a1 array shape= (1, 15)
```

In [69]:
```
# np.where
#the function returns the indices of elements in an input array where the give
a1=np.random.randint(1,100,15)
print("this is my a1 array",a1,"\n")
print("i use where function for getting greater than 50 element=",np.where(a1>
#np.where(condition,true,false)#syntax of where function
print(np.where(a1>50,0,a1))
np.where(a1%2==0,0,a1)
```

```
this is my a1 array [10 84 68 70 80 68 28 68 20 18 94 15 73 58  3]

i use where function for getting greater than 50 element= (array([ 1,  2,  3,
4,  5,  7, 10, 12, 13]),)
[10  0  0  0  0  0 28  0 20 18  0 15  0  0  3]
```

Out[69]:
```
array([ 0,  0,  0,  0,  0,  0,  0,  0,  0,  0,  0, 15, 73,  0,  3],
      dtype=int32)
```

In [73]:
```
# np.argmax   use to find larger/big element of array it find index of this el
np.argmax(a1)
np.argmin(a1)
```

Out[73]:
```
np.int64(14)
```

In [77]:
```
#np.cumsum  ise when we want to compute the cumulative sum of array element ov
print("for 1D function",np.cumsum(a1))
print("\nfor 2D...nD array\n",np.cumsum(a1,axis=0))
```

```
for 1D function [ 10  94 162 232 312 380 408 476 496 514 608 623 696 754 757]

for 2D...nD array
 [ 10  94 162 232 312 380 408 476 496 514 608 623 696 754 757]
```

In [82]:
```
# percentile  highier score and then back number
# use to compute the nth percentile of the geven data
#formula of percentile  (p=(n/N*100))
#n stand for number of values below x
# N stand for total count of population
```

```
print(np.percentile(a1,50))
print(np.median(a1))
```

```
68.0
68.0
```

In [88]:
```
# np.histogram    use for frequency of data distribution in the graphical form
# called as frequency chart show
c=np.random.randint(0,80,20)
print("this is my c array to check frequency of c=\n",c,"\n")
np.histogram(c,bins=[0,30,50,80])
```

```
this is my c array to check frequency of c=
 [14 16 64 58 76 49 15 75  2  0  8 39 77  9 36 34 48 54  2 44]
```

Out[88]:
```
(array([8, 6, 6]), array([ 0, 30, 50, 80]))
```

In [2]:
```
# np.corrcoef   return pearson product-moment correlation coefficients
# it varry in -1 to 1  "corelation coficant"
import numpy as np

salary = np.array([20000, 40000, 25000, 35000, 60000])
experience = np.array([1, 3, 2, 4, 2])

print(np.corrcoef(salary, experience)) # we can find n of corelation of array
```

```
[[1.         0.25344572]
 [0.25344572 1.        ]]
```

In [18]:
```
# np.isin
# we can see that one array having values are checked in a different numpy arr
c=np.random.randint(0,80,10)
print(c,"\n")
items=[10,20,30,40,50,60,70,80,90,100]
c[np.isin(c,items)]
```

```
[31 71 68  9 28 46  6 44 30 70]
```

Out[18]:
```
array([30, 70], dtype=int32)
```

In [32]:
```
#np.flip()
# use to switch the number last number to first
salary = np.array([20000, 40000, 25000, 35000, 60000])
print("this is without flip operation",salary)
print("use for one dimension",np.flip(salary))
b=np.random.randint(1,100,12).reshape(2,6)
print("this is 2D array",b)
print("all array flip row and column\n",np.flip(b))
print("only column flip=\n",np.flip(b,axis=1))
```

```
this is without flip operation [20000 40000 25000 35000 60000]
use for one dimension [60000 35000 25000 40000 20000]
this is 2D array [[ 6 82 42 17 78 73]
 [56 74 74 59 59 71]]
all array flip row and column
 [[71 59 59 74 74 56]
 [73 78 17 42 82  6]]
only column flip=
 [[73 78 17 42 82  6]
 [71 59 59 74 74 56]]
```

In [38]:

```
# np.put()
# add multipal value in array
print("my salary element[10000  5000 25000 35000 60000]")
np.put(salary,[0,1],[25000,1000])
print(salary)
```

```
my salary element[10000  5000 25000 35000 60000]
[25000  1000 25000 35000 60000]
```

In [39]:

```
# delete
print("my exesting salary [10000  5000 25000 35000 60000]")
np.delete(salary,1)#we insert index to delete
np.delete(salary,[0,2,3]) # for multipal delete value
```

```
my exesting salary [10000  5000 25000 35000 60000]
```

Out[39]:

```
array([25000, 25000, 35000, 60000])
```

In [52]:

```
"""set function
1=np.union
2=np.intersect
3=np.setdiff
4=np.setxor
5=np.in """
#1d use is compalsory

import numpy as np

m = np.array([1, 2, 3, 4, 5])
n = np.array([3, 4, 5, 6, 7])

print("union =", np.union1d(m, n))
print("intersect =", np.intersect1d(m, n))
print("set diff =", np.setdiff1d(m, n))
print("setxor it use to comman element present in array remove and give rest e
print("check element present in array =", np.in1d(m,5)) # we can also use isin
```

```
union = [1 2 3 4 5 6 7]
intersect = [3 4 5]
set diff = [1 2]
setxor it use to comman element present in array remove and give rest element =
[1 2 6 7]
check element present in array = [False False False False  True]
```

```
C:\Users\jivankakde\AppData\Local\Temp\ipykernel_18080\2870218284.py:17: Deprec
ationWarning: `in1d` is deprecated. Use `np.isin` instead.
  print("check element present in array =", np.in1d(m,5))
```

In [55]:
```
#np.clip  it use to limit to the array other element delete
limit= np.array([15, 5,7,25, 38, 100, 60,30,0,40])
print("my array element [15, 25, 38, 100, 60,30,0,40]")
np.clip(limit, 10, 50)
```

```
my array element [15, 25, 38, 100, 60,30,0,40]
```

Out[55]:
```
array([15, 10, 10, 25, 38, 50, 50, 30, 10, 40])
```

In [2]:
```

```

```
  Cell In[2], line 1
    numpy self =python numpy function + numpy array
          ^
SyntaxError: invalid syntax
```

In [ ]:
```

```